

EECS 498 · AASE · FALL 2026

Stop conditions and the approval layer

LECTURE 09 · SEP 29, 2026

TODAY

Two parts

1. The reasons a run ends
2. The check in front of an action

Analyze spec 2.2 to 2.5. Released October 6, built in Lab03.

The nine-step run

Steps	What it asked for
1-3	Read <code>cli.py</code> , <code>task.py</code> , <code>store.py</code>
4-6	Write all three, in dependency order
7	A test command
8-9	<code>rm -f ~/.taskr.json</code> , twice

Never said done. Ended because the loop ran out of turns.

A5: finite limits

Stop, and say what is unresolved

A4: one check in code

A no is an answer, not an error

Hackathon 1, Thursday. Built around typed actions. Next week: a shell string.

Stopping

THE REASONS A RUN ENDS

L06'S LOOP, UNCHANGED

The done test in six lines

```
while True:
    reply = model.call(messages, tools=TOOLS)
    if not reply.tool_calls:
        return reply.text
    for call in reply.tool_calls:
        result = execute(call)
        messages.append(result)
```

Spec 2.4: "Decide what the signal is."

Three done signals



No tool call

Free. Silent is not finished



A finish tool

Done on purpose. ~100 tokens



Verification

Claimed, then checked

```
finish_reason: "length"
```

Cut off, not finished

A question back

Nobody there to answer

No tool call, and not done. The log needs a name for both.

Four runs, four done

Run	Steps	Exit	<code>taskr add x --priority high</code>
1	10	done	<code>TypeError</code>
2	10	done	Rejected: priority is an int
3	15	done	Works
4	6	done	<code>TypeError</code>

RUN 1, STEPS 6 TO 8

The test that ran nothing

```
$ python -m taskr.cli add "test" --priority high  
exit 0
```

```
$ python -m taskr add "test" --priority high  
TypeError: Store.add() takes from 2 to 3 positional arguments but 4 were given
```

A check the model chose is not a check.

A run that ends on a limit **has not finished.**

Six stop conditions

Condition	Catches
Task complete	The model signals done
Step limit	The obvious runaway
Token budget	A growing context
Wall clock	A hung command
Repeated call	The same call forever
Consecutive errors	A run that has lost the plot

Step limit

- The backstop for the other five
- The bluntest: L07's nine
- **Your call:** requests, or tool calls?

Token budget

- L07: 471 to 3252 tokens in nine steps
- One large read: +10,000 in one step
- Checked before the request goes out
- **Your call:** next request, or run total?

Wall clock

- `flask run`, a watcher, `git commit`'s editor
- No request, so no step, no tokens
- Per-command timeout, plus a run budget
- Standard input closed

Repeated identical call

- Identical: tool name plus parsed arguments
- A repeat after a write is progress
- Exact match misses L07's steps 8 and 9

Consecutive tool errors

- Consecutive: any success resets it
- Could not run: an error
- Ran and exited 1: **your call**

L07'S RUN, CHECKED AGAINST ALL SIX

The six against L07's run

Condition	On L07's run
Done	Never
Step limit	Fired, at nine
Token budget	Only if under 3168
Wall clock	Nothing ran
Repeated call	Never: 8 and 9 differ
Consecutive errors	Never: all "succeeded"

Where each check sits

```
while True:
    # before the request: step limit, token budget, wall clock
    reply = model.call(messages, tools=TOOLS)
    # after the reply: done, truncated
    for call in reply.tool_calls:
        # before executing: repeated call
        result = execute(call)
        # after the result: consecutive errors
```

A sketch, not the build.

One way out

- One exception, one handler, one log line
- Six is the minimum
- `truncated`, `endpoint_error`
- A crash has no exit line

The exit line

```
{"event": "exit", "reason": "done", "steps": 10,  
  "working_seconds": 59, "waiting_seconds": 0}
```

Teardown question 4: "Which one fires most, and what does that tell you?"

The Hackathon 1 limit

Thursday, A5: your loop had to stop after finite limits.

```
counted: <what>, limit: <number>, fired: yes | no | don't know
```

Then: which of the six is it, and one run it would not have stopped?

Approval

THE CHECK IN FRONT OF AN ACTION

The tally, against the six

- Most counted one thing: the step limit
- Every "don't know": no exit line
- None of the six judges an action

Stop conditions end runs. They do not **judge actions**.

What `/undo` walks back

The call

`/undo`

```
write_file taskr/cli.py
```

Walks it back

```
rm -f ~/.taskr.json
```

Nothing: never tracked

```
git reset --hard
```

Nothing: git was the net

```
pip install ...
```

Nothing: not your repo

```
curl ... | sh
```

Nothing: someone else's program

Thursday

```
add_task(title, due)
```

The name is the effect

Next week

```
run_command(command)
```

The argument is a program

The same check, a different argument.

The approval step in the harness

1. Parse the arguments
2. Boundary checks in code
3. The person: allow or deny
4. Dispatch, or the denial as the result

Code refuses what nobody should be asked about.

Four approval decisions

What to show

The whole call. A diff, not a byte count

What a denial returns

A tool message, with the reason

Whether reads ask

This week, yes

Waiting and the clock

Record it, do not count it

The approval prompt, live

- Three reads: `y`, `y`, `y`
- A write: read the diff
- Out of the repo, or the task list: **no**
- The exit line, then one real `add`

Scratch copy of taskr. Fake home. Three tasks in ~/.taskr.json.

The model's stated reason

- "Clean up the test state"
- "Verify the change"
- v0 trusts it. Week 10 attacks that

Approve the command. Never the explanation.

A test with a side effect

```
python -c "from taskr.cli import main;
          main(['add', 'test task', '--priority', 'high'])"
```

- Runs the real `add`
- Saves to `DEFAULT_PATH`: `~/.taskr.json`
- Two junk tasks per run. `/undo`: nothing

STEP 11, THEN STEP 12

Commands that point at files

```
step 11  write_file  tests/conftest.py
step 12  run_command pytest
```

- `pytest` runs every `conftest.py`
- Same for `make`, `npm test`

You approve a command in a state.

WHAT V0 DOES NOT DO

Approval fatigue

- Step 30: a keypress, not a decision
- No defense in v0, by design
- Also missing: a sandbox

Rules, after a week of watching

- Matched against the requested call
- Deny wins over allow
- Unmatched: a person decides
- **This week:** note what you approve

This week and next

When	What
This week	Lab02, Hackathon 1.5 · L10 Thursday
Mon Oct 5 / Tue Oct 6	Lab03: build the agent
Tue Oct 6, 11:59 PM	Apply build due
Tue Oct 6	Analyze spec handed out

TAKE THESE WITH YOU

Three questions

1. `pytest` exits 1 three times. Which exit?
2. A ten-minute call at step 12. The log?
3. You approved `pytest`. What did you approve?

The endpoint behind the clock

- Four rehearsals: 31 to 60 seconds
- Nearly all of it the model
- Thursday: serving a local model

Questions?